Understanding Hardware-Assisted Memory Safety

An Experimental Study of ARM Memory Tagging and Software Sanitization

Suman Roy (SR No: 28164)

E0-256 Computer Systems Security
Indian Institute of Science, Bengaluru

November 2026

Motivation

- Stack and heap overflows, OOB accesses, use-after-free and double-free are still the root cause of most exploitable bugs in C/C++.
- **Software sanitizers** (ASan) are precise, but they are too slow and memory-hungry for production.
- **Hardware tagging** (ARM MTE): 4-bit tags on pointers and on 16-byte memory granules, checked by the CPU on every access.
- Noh et al., *ARM MTE Performance in Practice* (USENIX Security 2026), measured MTE's *cost* on real silicon.

This project: what does each mechanism actually *detect*, and at what cost?

Research questions

RQ1 **Detection coverage**: MTE vs. ASan on common spatial and temporal violations.

RQ2 **Detection boundaries**: what is caught by MTE only, by ASan only, or by neither?

RQ3 **Execution modes**: synchronous vs. asynchronous MTE, in reporting and performance.

RQ4 **Security–performance**: runtime and memory overhead across allocation-heavy and access-heavy workloads.

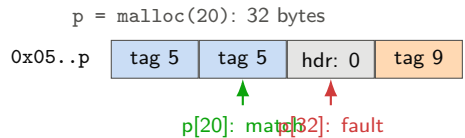
Background: two ways to check a memory access

AddressSanitizer (compiler + runtime)

- Shadow memory: 1 byte per 8 bytes, byte-precise.
- Poisoned *redzones* around objects.
- Freed memory goes to a *quarantine* (256 MiB).
- Every load and store is instrumented ⇒ deterministic.

ARM MTE (Armv8.5-A hardware)

- Pointer tag in bits 56–59; memory tag per 16-byte granule.
- `IRG` (random tag), `STG` (set tag), `LDG` (read tag).
- glibc tags heap chunks when `glibc.mem.tagging` is set; **no recompilation**.
- 16 tags ⇒ *probabilistic* detection.
- SYNC: precise fault. ASYNC: deferred, no address.



Starting point: Noh et al. (USENIX Security 2026)

- The paper measures Pixel 8/9 (Cortex-X, A7x, A5x), AmpereOne and Apple M5 on SPEC CPU, RocksDB, Nginx, PostgreSQL and Memcached.
- Their setup, which we reproduce: Clang/GCC binaries and glibc 2.36, with MTE enabled via `glibc.mem.tagging` on the *same binary*.
- Overheads are often modest, but with cliffs:
  - **6.64**× (456.hmmer, Pixel Performance core, SYNC): store serialization
  - **1.82**× even in ASYNC (Pixel Big core): tag-check structural hazard
  - 1.10× geomean on AmpereOne (SYNC); 1.43× from store-to-load forwarding
- Tagging large allocations (`STG`) is a visible cost.

They measure the cost; we measure what you get for it.

Experimental setup

Config	How	Platform
Baseline	`-fno-stack-protector -D_FORTIFY_SOURCE=0`	x86 + A64
Hardening	`-fstack-protector-strong -D_FORTIFY_SOURCE=3 -fsanitize=bounds-strict -fsanitize-trap=all`	x86 + A64
ASan	`-fsanitize=address`	x86-64
MTE-sync	baseline binary + `glibc.mem.tagging=3`	A64
MTE-async	baseline binary + `glibc.mem.tagging=1`	A64

- **x86-64**: Ryzen 5 5600H, native. Real timings for Baseline, Hardening and ASan.
- **AArch64 + MTE**: QEMU 10.2 user mode, `-cpu max`. Functional MTE; timings reflect *emulation*.
- A fault reporter in every test logs signal, `si_code` (sync 9 / async 8), address, PC → `file:line`.
- Baseline/Hardening on *both* platforms are controls: MTE vs. A64 baseline (same binary), ASan vs. x86 baseline.
- Detection: 20 runs per (program, config), **1,400 runs**. Performance: median time and peak RSS (5 runs), `perf stat` on x86-64.

Three methodology pitfalls we hit

1. **The compiler deletes the bugs.** At `-O1`/`-O2`, GCC removes `malloc; free; p[0]=42` and dead-buffer `strcpy` as UB.
   → The suite is built at `-O0` for every config.
2. **glibc 2.43's MTE is broken under QEMU.** `free()` raises a tag fault on *every* call, so every run is a false positive.
   → We use Debian's glibc **2.36**, the same version as the paper.
3. **ASan cannot run under QEMU user mode** (shadow reservation fails).
   → ASan is run natively on x86-64.

Phase 1: suite and boundary cases

Suite (proposal)

- stack overflow: 21 B into `char[10]`
- heap overflow: 21 B into `malloc(10)`
- OOB read / write: `arr[5]` of `int[5]`
- use-after-free: write after `free`
- double-free

Boundary cases (RQ2)

B1 write byte 12 of `malloc(10)`: inside the granule

B2 OOB index that jumps into another *live* object

B3 UAF after 320 MiB churn: chunk reused

B4 overflow `name[16]` $\rightarrow$ function pointer in the same `struct`, then call it

RQ1/RQ2: detection matrix (observed, 20 runs per cell)

	Baseline x86-64	Baseline AArch64	Hardening x86-64	Hardening AArch64	ASan x86-64	MTE-sync AArch64	MTE-async AArch64
Stack buffer overflow	! crash 0/20	✗ silent 0/20	✓ canary 20/20	✓ canary 20/20	✓ ASan 20/20	✗ silent 0/20	✗ silent 0/20
Heap buffer overflow	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✓ ASan 20/20	✓ tag fault 20/20	✓ async fault 20/20
Out-of-bounds read	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✓ ASan 20/20	✗ silent 0/20	✗ silent 0/20
Out-of-bounds write	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✓ ASan 20/20	✗ silent 0/20	✗ silent 0/20
Use-after-free	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✓ ASan 20/20	✓ tag fault 20/20	✓ async fault 20/20
Double-free	✓ glibc check 20/20	✓ glibc check 20/20	✓ glibc check 20/20	✓ glibc check 20/20	✓ ASan 20/20	✓ tag fault 20/20	✓ async fault 20/20
B1 Intra-granule overflow	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✓ ASan 20/20	✗ silent 0/20	✗ silent 0/20
B2 Far OOB into live object	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✓ tag fault 18/20	✓ async fault 18/20
B3 UAF after reallocation	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✗ silent 0/20	✓ tag fault 19/20	✓ async fault 18/20
B4 Intra-object overflow	✗ hijacked 0/20	✗ hijacked 0/20	✓ bounds trap 20/20	✓ bounds trap 20/20	✗ hijacked 0/20	✗ hijacked 0/20	✗ hijacked 0/20

✓ detected ! crashed without a report ✗ missed (program kept running) n/N = runs detected

RQ1: coverage on the suite

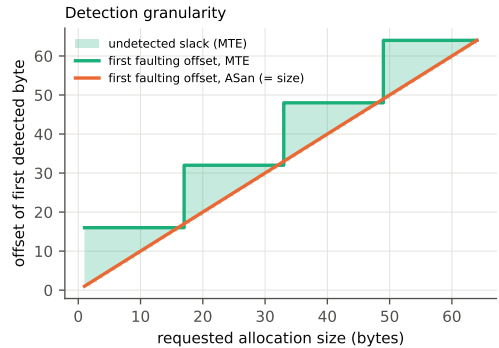
- **ASan: 6/6**, deterministic, exact line and bug class.
- **MTE: 3/6**: heap overflow, UAF and double-free, with no recompilation.
  - ✗ stack overflow: glibc tags only the heap; GCC has no stack tagging.
  - ✗ OOB read/write: `malloc(20)` is rounded to 32 B; `arr[5]` lies in the object's *own* padding. **Deterministic miss.**
- **Canary**: catches the stack overflow, but only at `return`, after the code has run on.
- **Baseline**: glibc's tcache already catches the double-free; the x86 stack overflow only *crashes*.

MTE's spatial resolution is 16 bytes, not 1 byte.

RQ2: the mechanisms fail on *different* bugs

Case	ASan	MTE
B1 intra-granule	✓ 20/20	✗ 0/20
B2 far OOB, live obj	✗ 0/20	✓ 18/20
B3 UAF after reuse	✗ 0/20	✓ 19/20
B4 intra-object	✗	✗

- B4 hijacks control flow in every run; only `-fsanitize=bounds-strict` traps it.
- Memory tagging is **not** a CFI substitute (Topic 5).



RQ3: synchronous vs. asynchronous MTE

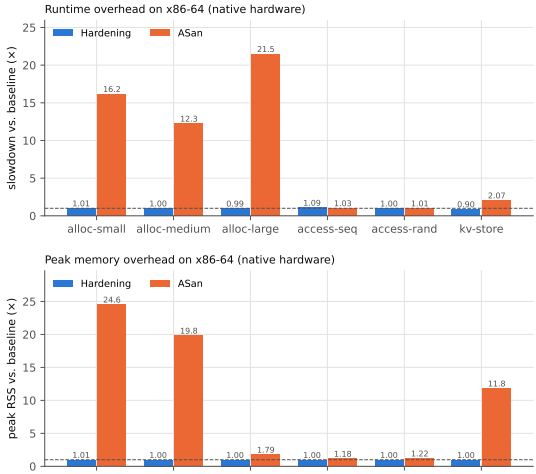
Probe: one illegal store (or 16 illegal loads) into a neighbouring "secret" object, then a syscall-free loop.

Access	Mode	fault address	store committed?	leaked bytes used
store	SYNC	exact	no	–
store	ASYNC	0	**yes**	–
load	SYNC	exact	–	0
load	ASYNC	0	–	**1**

- Both modes detect the same suite bugs; ASYNC reports at the next library call (`puts@plt`).
- On hardware the ASYNC report waits for the next *kernel entry*: a longer window.
- Emulated runtime: ASYNC $\approx 1.01\times$ SYNC. Not meaningful; the paper shows SYNC cliffs of up to $6.64\times$.

ASYNC detects after the fact; it does not prevent the write.

RQ4: native overhead (x86-64)



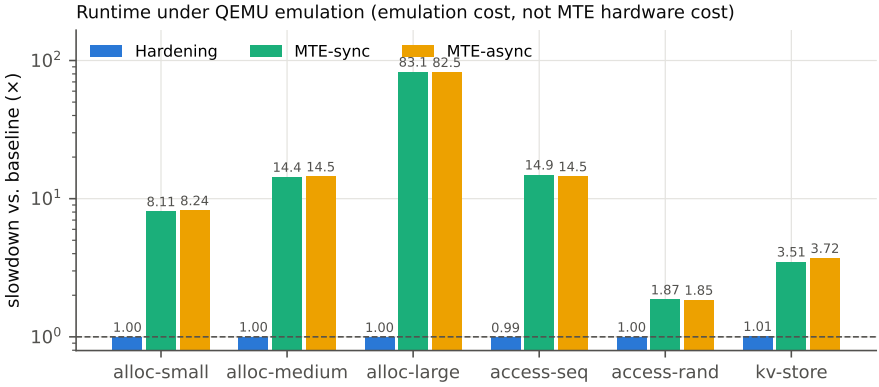
ASan cost is driven by allocation (redzones, poisoning, quarantine), not by checking each access: memory-bound loops cost only $\approx 1.0\times$.

RQ4: why ASan costs what it costs (`perf stat`, x86-64)

	Hard.	ASan relative to baseline				
Workload	instr.	instr.	cycles	IPC (base→ASan)	cache-miss	page-faults
alloc-small	1.00×	20.2×	16.0×	1.59→2.02	1,003×	1,266×
alloc-medium	1.00×	5.0×	12.7×	1.72→0.68	61×	158×
alloc-large	1.00×	7.8×	21.6×	0.83→0.30	78×	8×
access-seq	1.00×	1.8×	1.0×	1.04→1.83	1×	1×
access-rand	1.00×	1.9×	1.0×	0.03→0.05	1×	1×
kv-store	1.02×	3.4×	2.3×	0.83→1.24	2×	16×

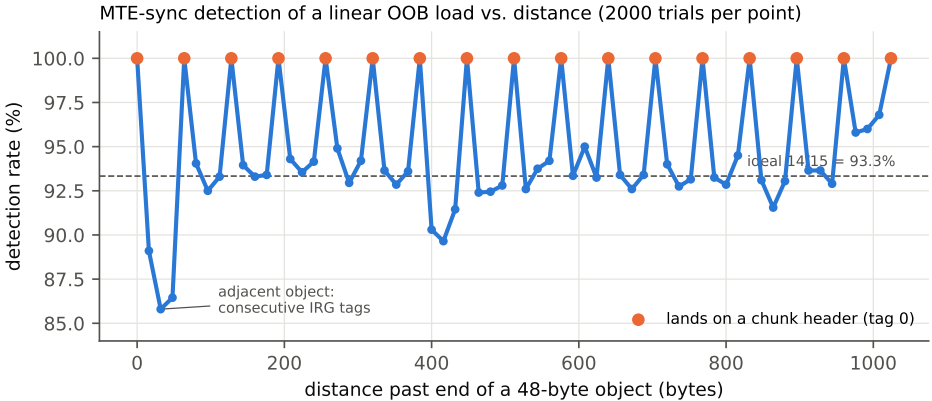
- **Allocation-heavy:** up to 20× instructions, 1,266× page faults, IPC drops (0.83→0.30): quarantine and shadow poisoning are memory traffic.
- **Memory-bound:** 1.77× instructions but 1.00× cycles: shadow checks hide behind DRAM stalls.
- **Hardening:** 1.00–1.02× instructions; its timing differences are noise.

RQ4: AArch64 under QEMU: where MTE adds work



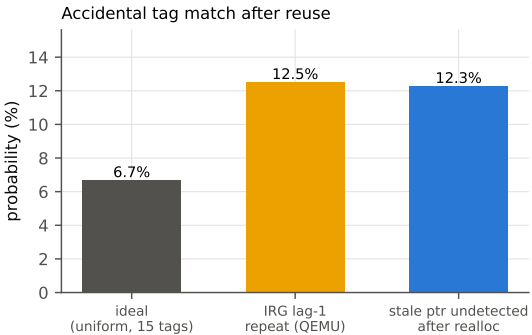
- This is emulation cost: QEMU checks tags in software.
- The relative pattern is still informative: cost grows with **bytes tagged per allocation** (`STG` on every `malloc`/`free`), matching the paper's large-allocation finding (§4.4.4).

Phase 4: the finite tag space, detection vs. distance



- Chunk headers carry tag 0 (never assigned to user pointers): **100%**.
- Far objects: **93.4%**, close to the ideal 14/15.
- Adjacent object: only **85.8–89.1%**.

Phase 4: accidental tag matches after reuse



- Freed memory is re-tagged to 0, so UAF before reuse: 100% detected.
- After reallocation: **12.3%** undetected (40,000 trials), against 6.7% ideal.
- Cause: consecutive `IRG` tags repeat **12.5%** of the time in QEMU.
- The same correlation explains the weak adjacent-object rate.

The guarantee is only as good as the tag generator.

Security–performance trade-off

	Detects	Misses	Cost
Hardening	stack smash (at return), index OOB	all heap bugs	$\approx 1.0\times$
ASan	all 6 suite bugs, byte-precise	far OOB, UAF after quarantine	1.01–21.5$\times$ time, up to 24.6$\times$ RSS
MTE-sync	heap OOB beyond granule, UAF, double-free, far OOB, reuse	stack, sub-granule, intra-object	3.1% tag storage; hw: modest + cliffs
MTE-async	same, *after* the access	+ no address	cheaper on hw (paper)

- The mechanisms are **complementary**: ASan for testing, MTE for production, plus stack protection and CFI.

Course mapping, limitations, conclusion

Course topics

- T1–2: stack smashing, the suite
- T3: software vs. hardware defenses
- T4: redzones/shadow vs. 16-byte tags
- T5: B4 shows MTE $\neq$ CFI
- T16: MTE instructions, modes, glibc/kernel integration

Limitations

- No MTE silicon: QEMU validates behaviour, not timing
- ASan on x86-64; no stack tagging (GCC); ASYMM not evaluated

Conclusion

- ASan: deterministic, byte-precise, too costly for production.
- MTE: no recompilation; catches the long-range and temporal bugs ASan misses, probabilistically and at 16-byte resolution.
- ASYNC trades prevention for detection.
- Neither stops an intra-object control-flow hijack.

Thank you

Questions?

All code, data and scripts: `memory-safety-eval` repository (`./run_all.sh`)